

26. Реализация транслятора, совмещённого с нисходящим анализатором.

Нисходящий транслятор

Если построенная схема трансляции пригодна для нисходящего анализа, то можно построить нисходящий **транслятор** – программу, осуществляющую нисходящий синтаксический анализ параллельно с вычислением атрибутов, в соответствии с правилами данной схемы трансляции.

Нисходящий транслятор представляет собой набор функций, взаимно однозначно соответствующих нетерминалам порождающей грамматики. Эти функции рекурсивно вызывают друг друга и имеют общую глобальную переменную `lookahead`, значение которой указывает на текущий символ входной строки в процессе ее разбора.

Обработка входной строки транслятором производится путем установки указателя `lookahead` на первый символ строки и вызова функции `S`, соответствующей аксиоме грамматики.

примечание

Что такое нисходящий транслятор?

Это программа, которая одновременно:

- Разбирает синтаксис (нисходящий LL(1)-анализ)
- Выполняет семантические действия (вычисляет атрибуты, генерирует код, заполняет таблицы)

Ключевые свойства:

- Функциональная структура: одна функция на каждый нетерминал
- Работает за один проход (синтаксис + семантика вместе)
- Использует общую переменную `lookahead` для доступа к текущему токenu

Важные ограничения

Только LL(1)-грамматики — иначе `CHOOSE(A, lookahead)` не сможет однозначно выбрать правило

Схема трансляции должна быть пригодна — обычно это L-атрибутные грамматики

Левой рекурсии быть не должно — иначе рекурсивные вызовы заикнутся

Алгоритм функции для нетерминала

Вход: схема трансляции, основанная на LL(1)-грамматике

$$G = (\Sigma, \Gamma, P, S),$$

нетерминал A .

Выход: код функции для A .

Функция $TYPE(x)$ возвращает тип x , функция $CHOOSE(A, a)$ возвращает «атрибутный» вариант правила вывода, содержащегося в клетке $[A, a]$ таблицы нисходящего анализатора грамматики G (предполагается, что таблица построена предварительно). Построенная для нетерминала функция имеет вид

$$A(i_1 : TYPE(A.i_1), \dots, i_k : TYPE(A.i_k)) : TYPE(A.s)$$

```
1  цикл по символам из  $\{X \mid \exists \alpha, \beta : (A \rightarrow \alpha X \beta) \in G\}$ 
2      цикл по атрибутам  $X.i_j$  ( $X.s$ )
3          объявить локальную переменную  $x_{i_j}$  (соотв.  $x_s$ )
4  объявить локальный массив  $A\_develop[ ]$ 
```

```
5  $A\_develop[ ] = \text{CHOOSE}(A, lookahead)$ 
6 если ( $A\_develop == \text{error}$ )  $A = \text{error}$ ; выход
7  $i = 1$ 
8 цикл пока  $i \leq |A\_develop|$ 
9      $Y = A\_develop[i]$ 
10     если ( $Y \in \Sigma$ )
11         если ( $Y = lookahead$ )
12              $y_s = Y.s$ ;  $lookahead = \text{next}$ 
13         иначе  $A = \text{error}$ ; выход
14     иначе если ( $Y \in \Gamma$ )
15          $y_s = Y(y_{i_1}, \dots, y_{i_m})$ 
16     иначе
17         выполнить действие, заменив атрибуты
            переменными для них
18      $i++$ 
```

пример из шура (стр. 210)

Завершая параграф, рассмотрим конкретный пример построения рекурсивной функции для нетерминала.

Пример 2.5. Рассмотрим функцию для нетерминала E' схемы трансляции TGA_3 из примера 2.3. Наследуемый и синтезируемый атрибуты этого символа являются числами. Значения функции SNOOSE нам известны из таблицы МП-автомата для соответствующей грамматики (ч. 2, пример 2.6): нетерминал E' разворачивается как $+TE'_1$, если очередной символ входной строки равен $'+'$ и аннулируется, если этот символ равен $')$ ' или концу строки; в остальных случаях выдается сообщение об ошибке. Мы соответственно адаптируем конструкцию, изложенную выше. Заметим также, что вызываемая в строке 4 процедура T не имеет параметров, так как нетерминал T не имеет наследуемых атрибутов.

$E' (i : \text{int}) : \text{int}$

1. объявить $t, i1, s1 : \text{int}$
2. %% тело функции
3. **если** $lookahead = '+'$
4. $t \leftarrow T$
5. $i1 \leftarrow i + t$

6. $s1 \leftarrow E'(i1)$
7. $E' \leftarrow s1$
8. **иначе если** $lookahead \in \{), \$ \}$
9. $E' \leftarrow i$
10. **иначе** $E' \leftarrow \text{error}$